

Monthly Project Progress Report

Project Overview

This month, our main focus was to **build and automate a complete data annotation and object detection system** — covering every step from **data collection** to **model training, tracking, and deployment**.

Key Achievements

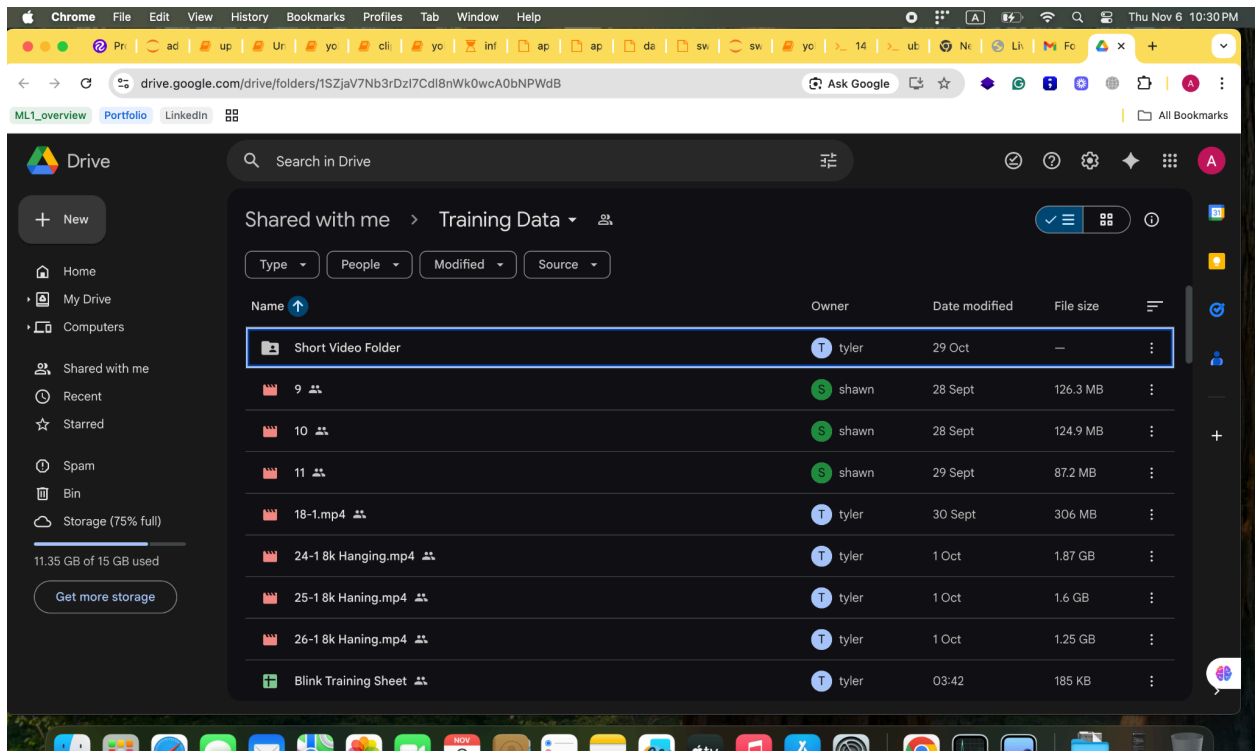
- Large-scale data annotation and model training
 - Full server setup and GPU environment configuration
 - Web API development with live camera inference
 - Automation of data workflows using APIs
 - Research on automatic AI-based annotation
-

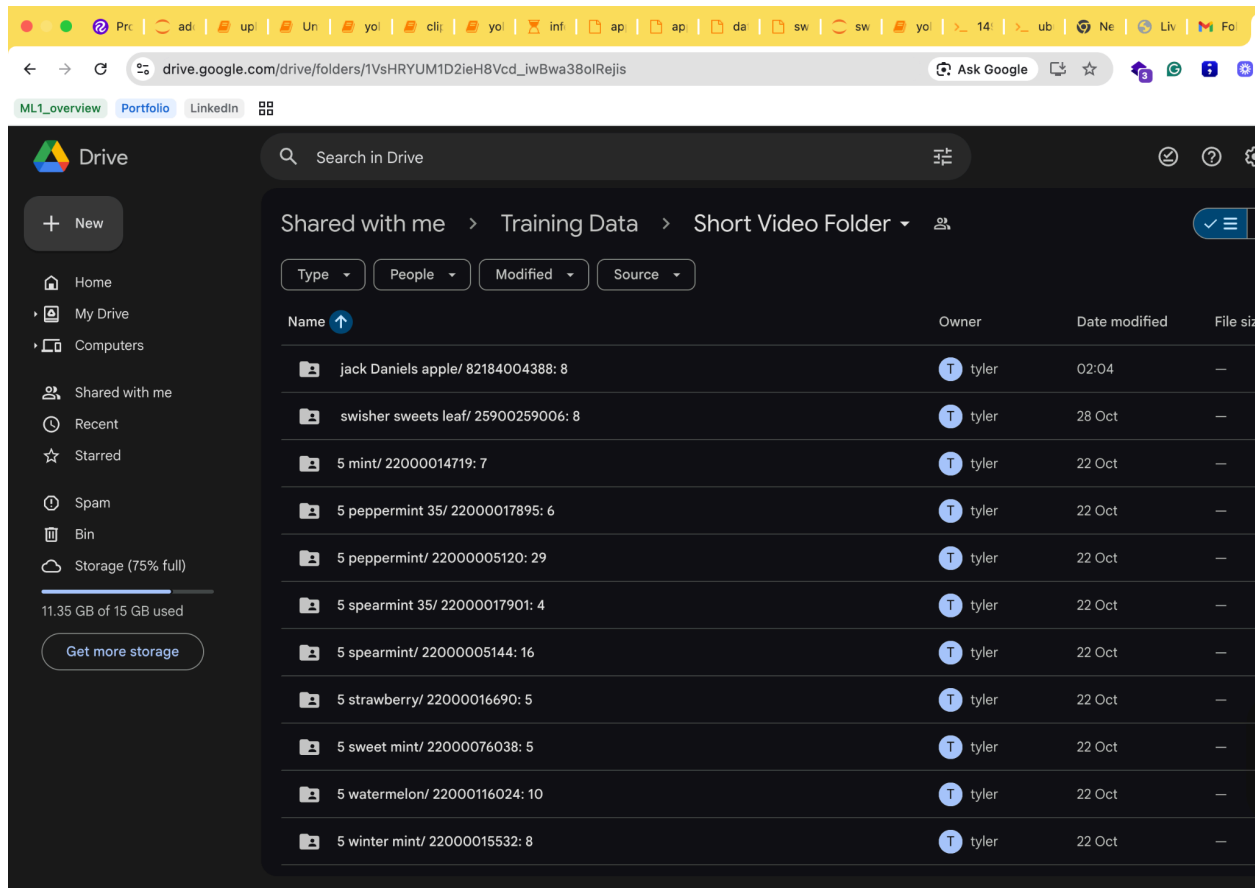
1. Data Preparation and Annotation

1.1 Initial Data Handling

- Received raw datasets (images and videos) with inconsistent formats.
- Reformatted and cleaned data to make it ready for annotation and training.

- Organized directory structure and verified labeling consistency.

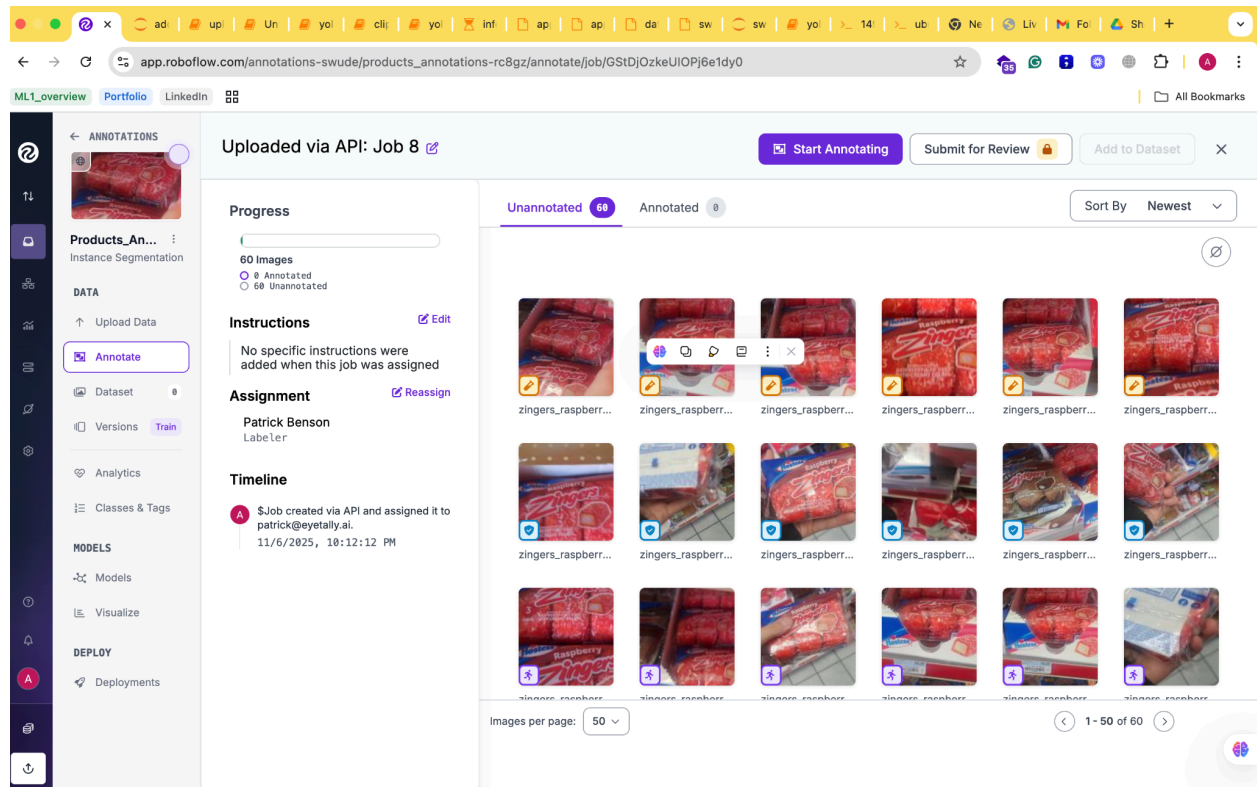


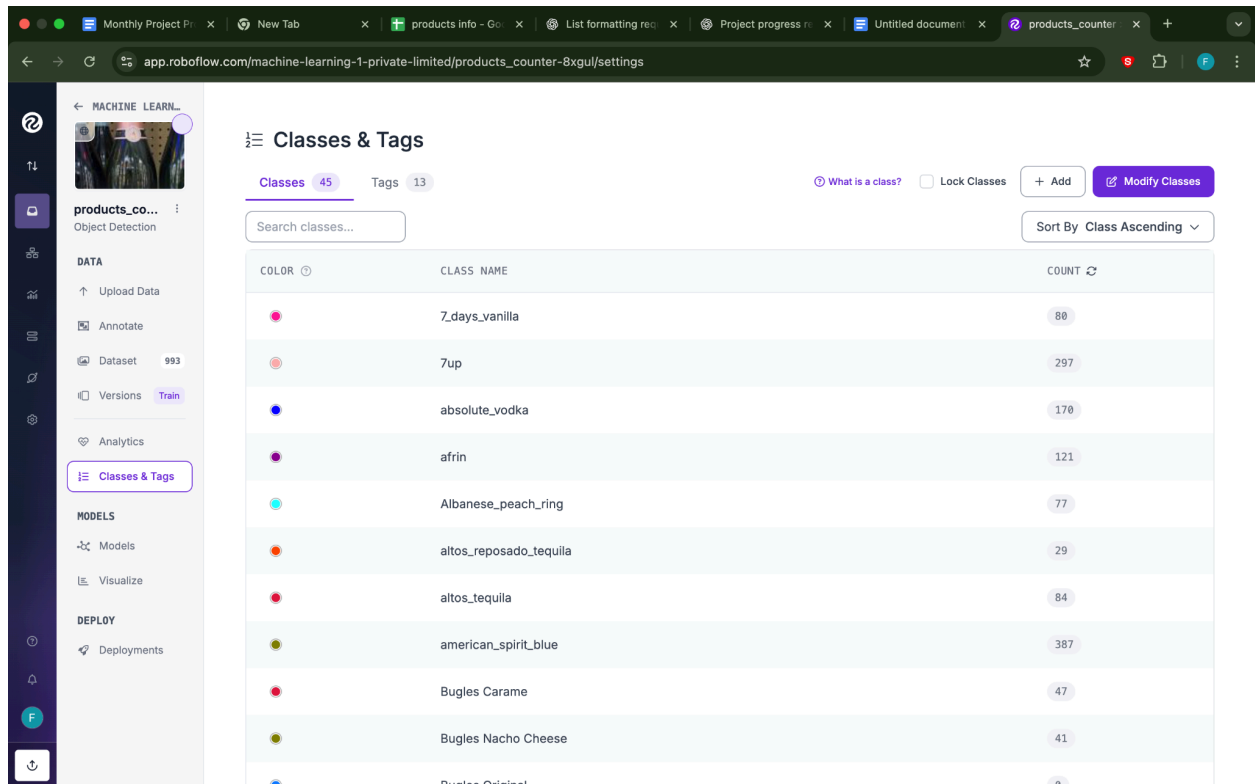


1.2 Manual Annotation

- Manually annotated **48 different product types**.
- Used **video frames** to mark each product accurately.
- Maintained strict quality control to ensure precise labeling.
- **Patrick Benson** joined the annotation process and completed **4 product types**.
- I worked half-time on annotation and half-time on automation and scripting to simplify our workflow.
- In total, I personally completed **44 product annotations** this month.

- Together, our team achieved **48 completed annotations**.





We have done these types

1. 19_crimes_sauignon_blance
2. 19_crimes_red_frame
3. 19_crime_rose_frame
4. American_Spirit_Black
5. 7_days_vanilla
6. 7up
7. Albanese_peach_ring
8. Bugles Caramé
9. Bugles Nacho Cheese
10. Cheez it Duoz Cheddar Jack Baby Swiss
11. Cheez it Pepper Jack
12. Cheez it Snack Mix Double Cheese
13. Cheez it Snapd Extra Crunchy
14. Chex Mix Remix Buffalo Sandwich
15. Chips Ahoy Mini
16. Corn Nuts BBQ
17. Corn Nuts Chile Picante Con Limon
18. Corn Nuts Kickin Dill Pickle
19. Corn Nuts Loaded Taco
20. Corn Nuts Mexican Style Street Corn Flavor
21. Corn Nuts Nuts Original

22. Corn Nuts Ranch
23. Dots Pretzels Parmesan Garlic
24. Gardettos Chipotle Cheddar
25. Gold Fish Cheddar
26. Gold Fish Flavor Blasted Xtra Cheddar
27. Hanover Pretzel Dips Milk Chocolate
28. Hanover Pretzel Dips White Creme
29. Kettle Krinkle Cut Dill Pickle
30. Kettle Krinkle Cut Habanero Lime
31. Kettle Krinkle Cut Salt Pepper
32. Kettle Potato CHips Backyard Barbeque
33. Kettle Potato Chips Honey Dijon
34. Kettle Potato Chips Jalapeno
35. Kettle Potato Chips Sea Salt
36. Oreo Muddu Buddies Cookies
37. absolute_vodka
38. afrin
39. altos_reposado_tequila
40. altos_tequila
41. american_spirit_blue
42. gruet
43. redbull
44. sea_glass
45. swisher_sweets_leaf
46. twisted_tes_extreme

1.3 Tools and Workflow

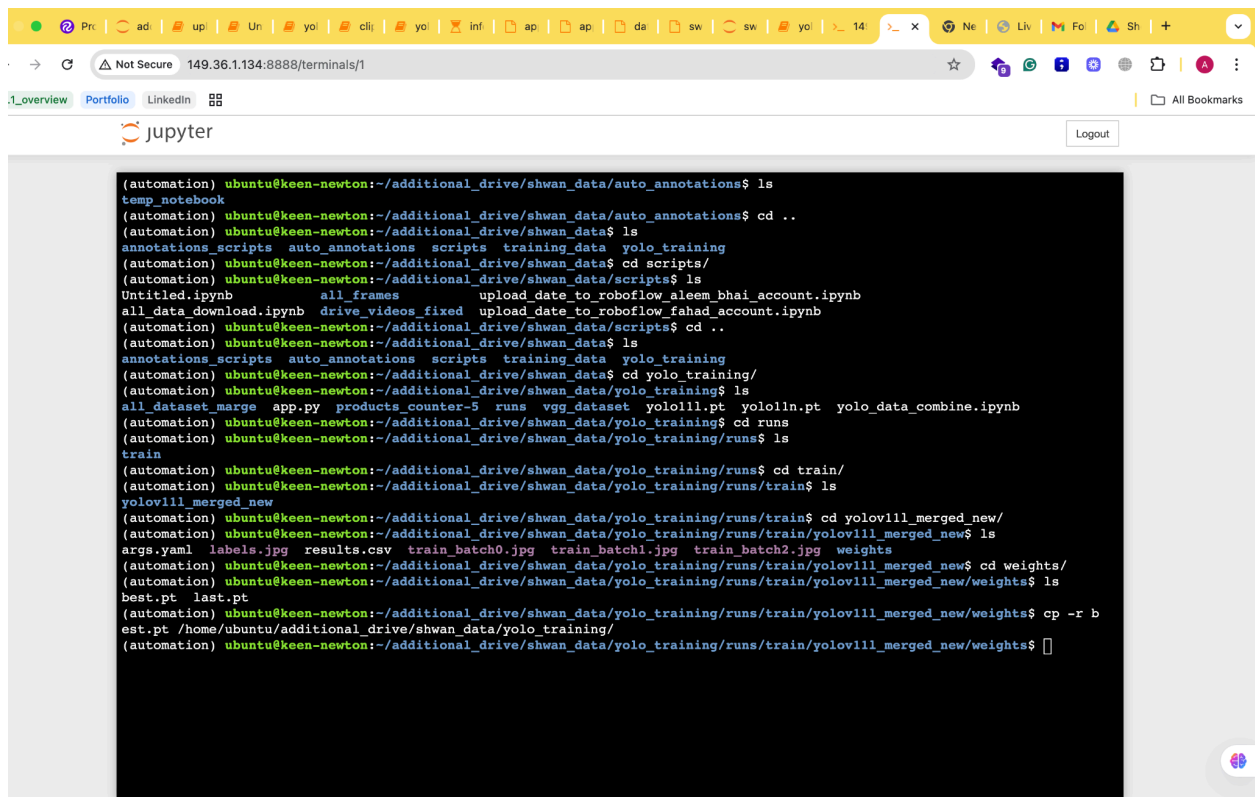
- Used **Roboflow** for dataset management and annotation consistency.
- Verified all annotations visually and statistically to minimize errors.

2. Server Setup and Environment Configuration

2.1 Server Initialization

After receiving the new server, we successfully set up:

- Ubuntu environment
- Conda environments for training, automation, and inference
- Installed required libraries (Ultralytics, OpenCV, Flask, PyTorch, etc.)



```
(automation) ubuntu@keen-newton:~/additional_drive/shwan_data/auto_annotations$ ls
tmp notebook
(automation) ubuntu@keen-newton:~/additional_drive/shwan_data/auto_annotations$ cd ..
(automation) ubuntu@keen-newton:~/additional_drive/shwan_data$ ls
annotations scripts auto_annotations scripts training_data yolo_training
(automation) ubuntu@keen-newton:~/additional_drive/shwan_data$ cd scripts/
(automation) ubuntu@keen-newton:~/additional_drive/shwan_data/scripts$ ls
Untitled.ipynb all frames upload date_to roboflow_aleem_bhahi account.ipynb
all data download.ipynb drive videos fixed upload date_to roboflow_fahad_account.ipynb
(automation) ubuntu@keen-newton:~/additional_drive/shwan_data/scripts$ cd ..
(automation) ubuntu@keen-newton:~/additional_drive/shwan_data$ ls
annotations scripts auto_annotations scripts training_data yolo_training
(automation) ubuntu@keen-newton:~/additional_drive/shwan_data$ cd yolo_training/
(automation) ubuntu@keen-newton:~/additional_drive/shwan_data/yolo_training$ ls
all_dataset_merge app.py products counter-5 runs vgg_dataset yolov11.pt yolov11n.pt yolo_data_combine.ipynb
(automation) ubuntu@keen-newton:~/additional_drive/shwan_data/yolo_training$ cd runs
(automation) ubuntu@keen-newton:~/additional_drive/shwan_data/yolo_training/runs$ ls
train
(automation) ubuntu@keen-newton:~/additional_drive/shwan_data/yolo_training/runs$ cd train/
(automation) ubuntu@keen-newton:~/additional_drive/shwan_data/yolo_training/runs/train$ ls
yolov11l_merged_new
(automation) ubuntu@keen-newton:~/additional_drive/shwan_data/yolo_training/runs/train$ cd yolov11l_merged_new/
(automation) ubuntu@keen-newton:~/additional_drive/shwan_data/yolo_training/runs/train/yolov11l_merged_new$ ls
args.yaml labels.jpg results.csv train_batch0.jpg train_batch1.jpg train_batch2.jpg weights
(automation) ubuntu@keen-newton:~/additional_drive/shwan_data/yolo_training/runs/train/yolov11l_merged_new$ cd weights/
(automation) ubuntu@keen-newton:~/additional_drive/shwan_data/yolo_training/runs/train/yolov11l_merged_new/weights$ ls
best.pt last.pt
(automation) ubuntu@keen-newton:~/additional_drive/shwan_data/yolo_training/runs/train/yolov11l_merged_new/weights$ cp -r b
est.pt /home/ubuntu/additional_drive/shwan_data/yolo_training/
(automation) ubuntu@keen-newton:~/additional_drive/shwan_data/yolo_training/runs/train/yolov11l_merged_new/weights$
```

2.2 Environment Optimization

- Configured GPU acceleration using **CUDA** for faster training.
- Verified **CUDA**, **cuDNN**, and **PyTorch** compatibility.
- Created separate environments for:

- Model Training
- Automation Scripts
- Web Deployment

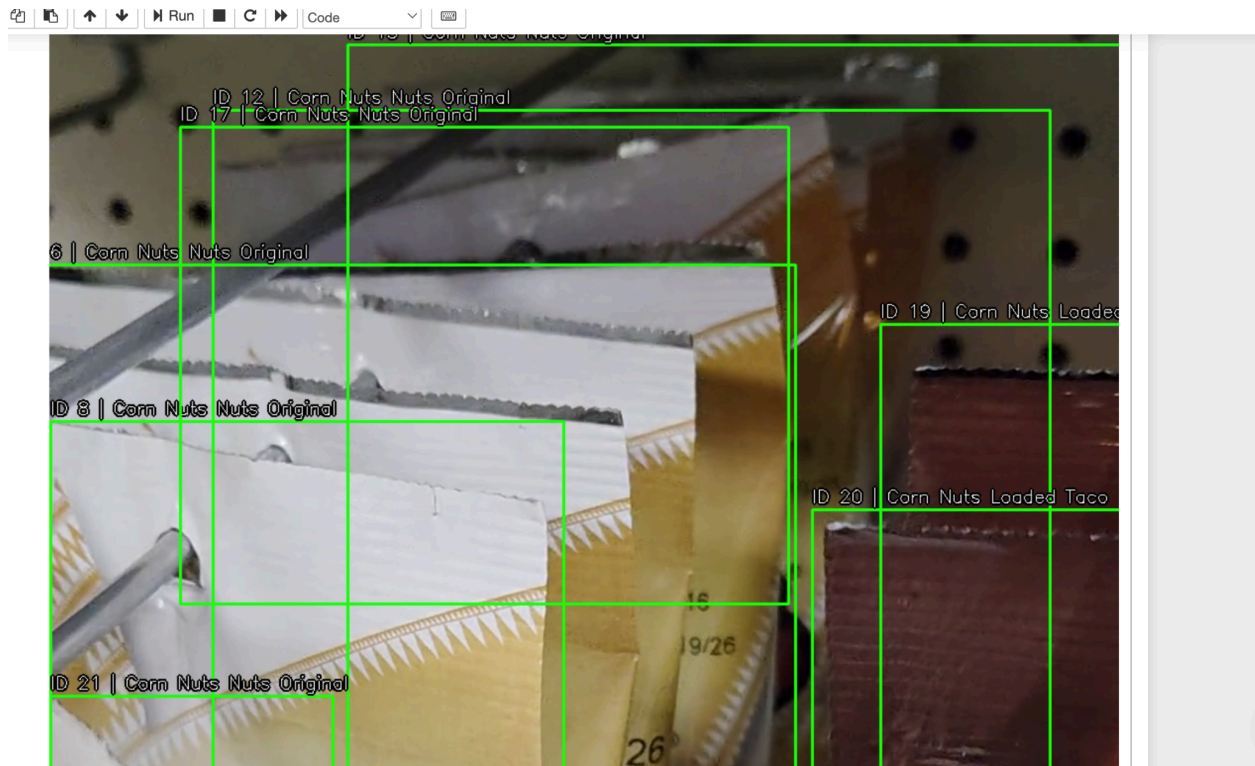
3. Object Detection Model Development

3.1 YOLO Model Training

We trained multiple YOLO models for product detection:

- YOLOv8
- YOLOv11 (latest version)

s	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 83%	5/6 2.5it/s 1.7s
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	6/6 3.3it/s 1.8
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	6/6 3.3it/s 1.8
	all	183	818	0.885	0.784	0.839	0.748	
s	Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	
	155/300	10.3G	0.4146	0.328	0.9307	34	640: 100%	170/170 2.4it/s 1:11
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 17%	1/6 0.7it/s 0.4s
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 33%	2/6 1.2it/s 0.9s
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 50%	3/6 1.4it/s 1.4s
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 67%	4/6 1.8it/s 1.8s
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 83%	5/6 2.0it/s 2.2s
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	6/6 2.5it/s 2.4
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	6/6 2.5it/s 2.4
	all	183	818	0.838	0.789	0.836	0.751	
	Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	
	156/300	10.3G	0.4095	0.3277	0.9333	41	640: 100%	170/170 2.4it/s 1:10
s	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 17%	1/6 0.9it/s 0.3s
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 33%	2/6 1.6it/s 0.6s
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 50%	3/6 2.0it/s 1.0s
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 67%	4/6 2.3it/s 1.3s
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 83%	5/6 2.5it/s 1.7s
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	6/6 3.3it/s 1.8
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	6/6 3.3it/s 1.8
	all	183	818	0.858	0.802	0.838	0.757	
	Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	
	157/300	10.4G	0.4055	0.3192	0.9324	40	640: 100%	170/170 2.4it/s 1:11
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 17%	1/6 0.9it/s 0.3s
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 33%	2/6 1.6it/s 0.7s
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 50%	3/6 2.0it/s 1.0s
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 67%	4/6 2.3it/s 1.3s
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 83%	5/6 2.5it/s 1.7s
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	6/6 3.3it/s 1.8
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	6/6 3.3it/s 1.8



3.2 Model Improvement

- Added new annotated data to improve model accuracy.
- Retrained YOLO models and observed noticeable improvements.
- Evaluated on test datasets for performance reliability.

3.3 Results

- Improved **mAP (Mean Average Precision)** after new data inclusion.
- Models performed consistently on unseen videos and live tests.

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size	
160/300	10.3G	0.4072	0.3243	0.9314	48	640: 100%	170/170 2.4it/s 1:11
Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 17%	1/6 0.8it/s 0.4s
Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 33%	2/6 1.3it/s 0.8s
Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 50%	3/6 1.8it/s 1.1s
Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 67%	4/6 2.1it/s 1.5s
Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 83%	5/6 2.4it/s 1.8s
Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	6/6 3.1it/s 1.9
Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	6/6 3.1it/s 1.9
all		183	818	0.823	0.788	0.837	0.75
Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size	
161/300	10.3G	0.4084	0.3255	0.9344	49	640: 100%	170/170 2.4it/s 1:10
Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 17%	1/6 0.9it/s 0.3s
Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 33%	2/6 1.5it/s 0.7s
Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 50%	3/6 1.9it/s 1.0s
Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 67%	4/6 2.2it/s 1.4s
Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 83%	5/6 2.4it/s 1.7s
Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	6/6 3.0it/s 2.0
Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	6/6 3.0it/s 2.0
all		183	818	0.858	0.794	0.835	0.749
Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size	
162/300	10.3G	0.3987	0.3189	0.9281	23	640: 100%	170/170 2.4it/s 1:11
Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 17%	1/6 0.9it/s 0.3s
Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 33%	2/6 1.6it/s 0.6s
Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 50%	3/6 2.1it/s 1.0s
Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 67%	4/6 2.4it/s 1.3s
Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 83%	5/6 2.6it/s 1.6s
Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	6/6 3.4it/s 1.8
Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	6/6 3.4it/s 1.8
all		183	818	0.876	0.762	0.837	0.749
Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size	
163/300	10.3G	0.3994	0.3085	0.9288	22	640: 100%	170/170 2.4it/s 1:10

4. Product Tracking and Counting System

4.1 Pipeline Development

Built a real-time product tracking and counting system using:

- Deep SORT
- ByteTrack

Both algorithms were tested for stability and ID consistency.

```

print(f"Saved annotated video to: {os.path.abspath(output_path)}")
print(f"Total counted (persisted) objects: {len(seen_ids_global)}")
for cls_name, ids in sorted(seen_ids_per_class.items(), key=lambda x: x[0]):
    print(f" {cls_name}: {len(ids)}")

```

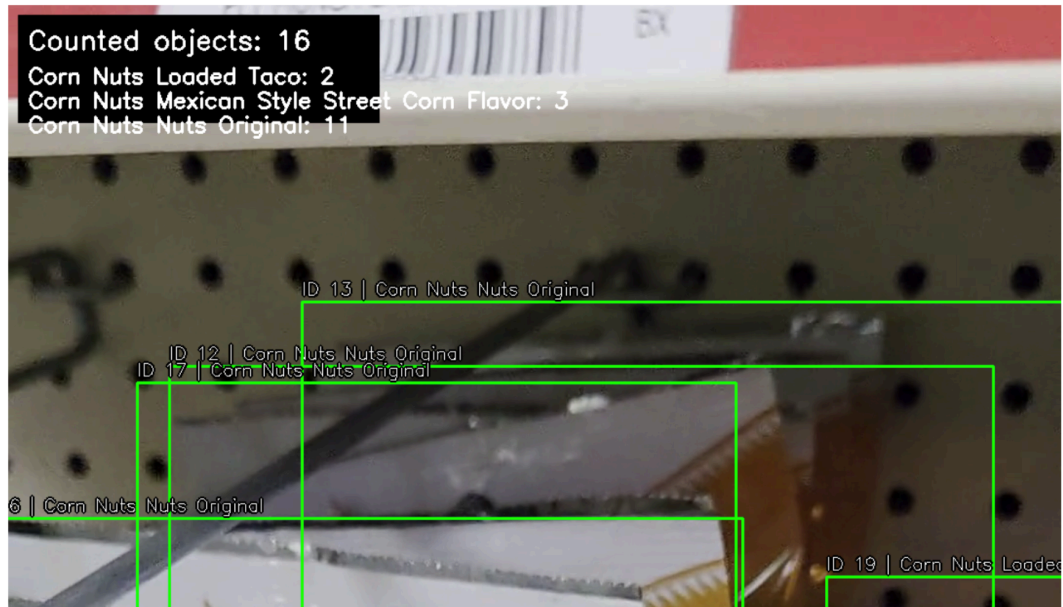
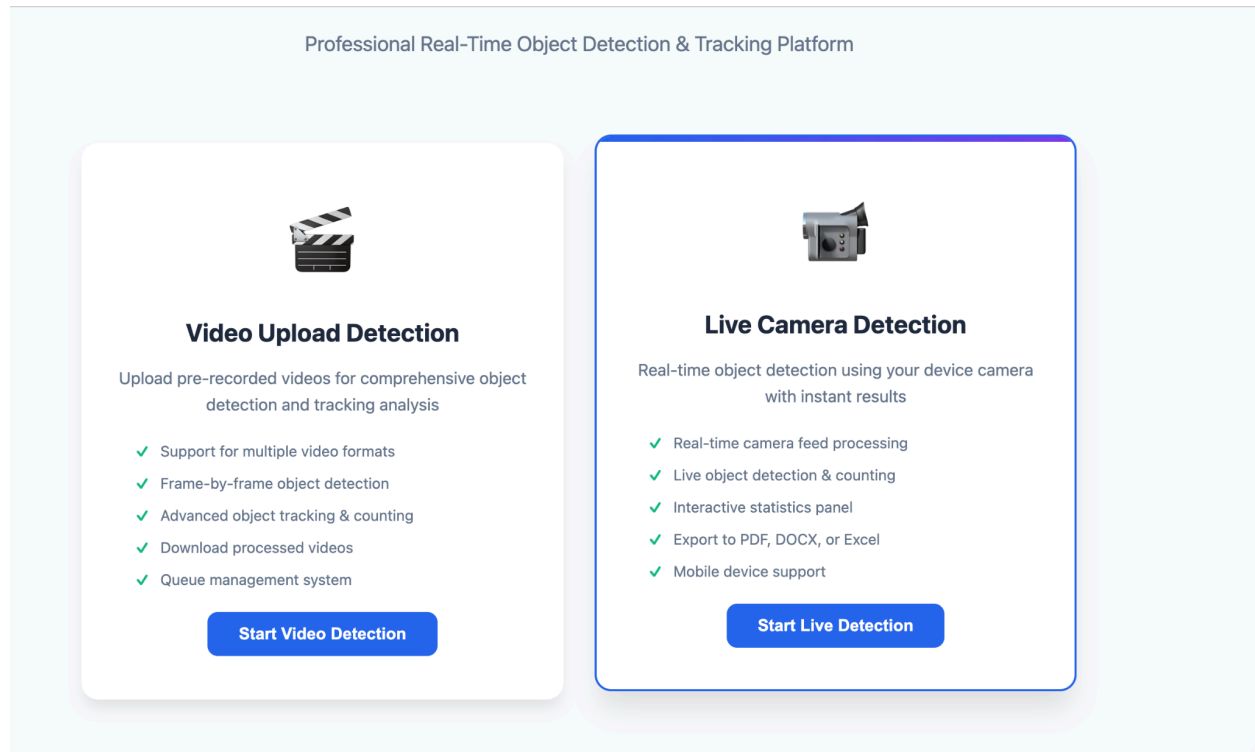


image 1/1 /home/ubuntu/additional_drive/shwan_data/yolo_training/all_dataset_marge/images/val/19_crimes_sauignon_bl
 ance_frame_39.png: 640x384 2 19_crimes_sauignon_blances, 27.4ms
 Speed: 2.3ms preprocess, 27.4ms inference, 35.0ms postprocess per image at shape (1, 3, 640, 384)



4.2 Testing and Evaluation

- Tested pipelines on recorded videos and live camera feeds.
- Verified accurate product counting and tracking behavior.



5. Web Application and API Integration

5.1 Flask-based Inference API

Developed a **Flask web API** for easy model inference:

- Upload Image → Get JSON Response
- Added support for video and real-time inference

Professional Real-Time Object Detection & Tracking Platform



Video Upload Detection

Upload pre-recorded videos for comprehensive object detection and tracking analysis

- ✓ Support for multiple video formats
- ✓ Frame-by-frame object detection
- ✓ Advanced object tracking & counting
- ✓ Download processed videos
- ✓ Queue management system

[Start Video Detection](#)

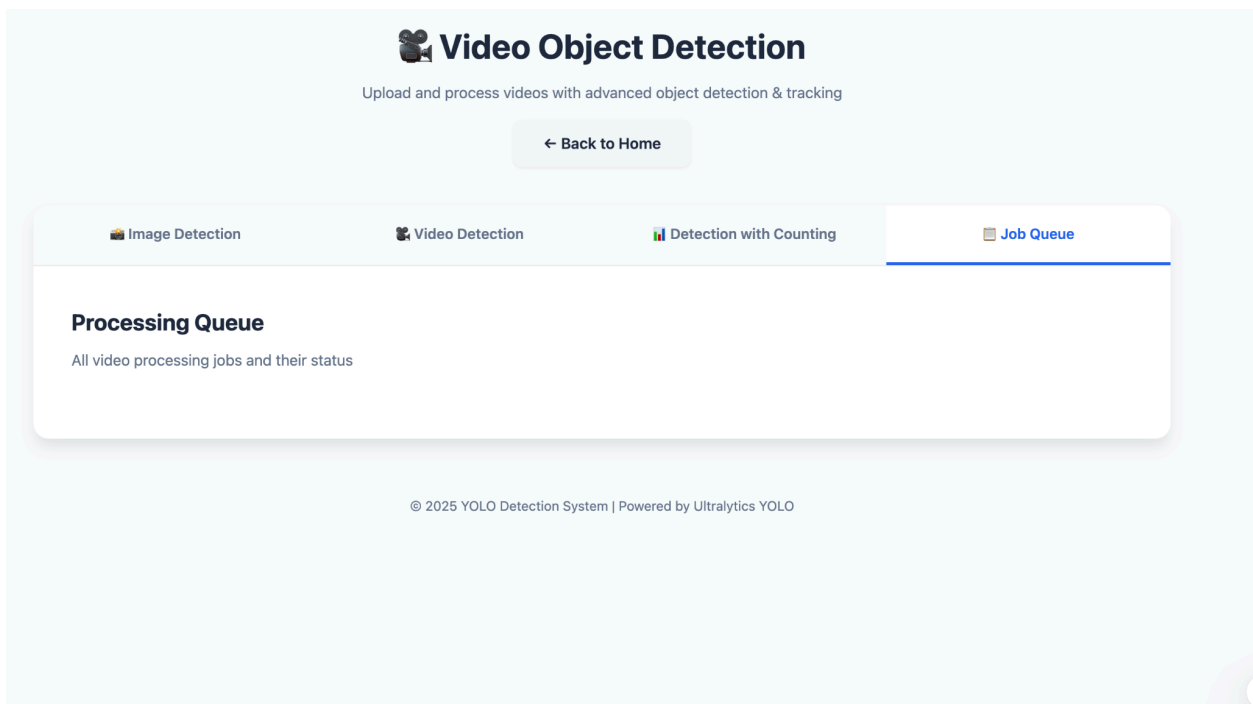
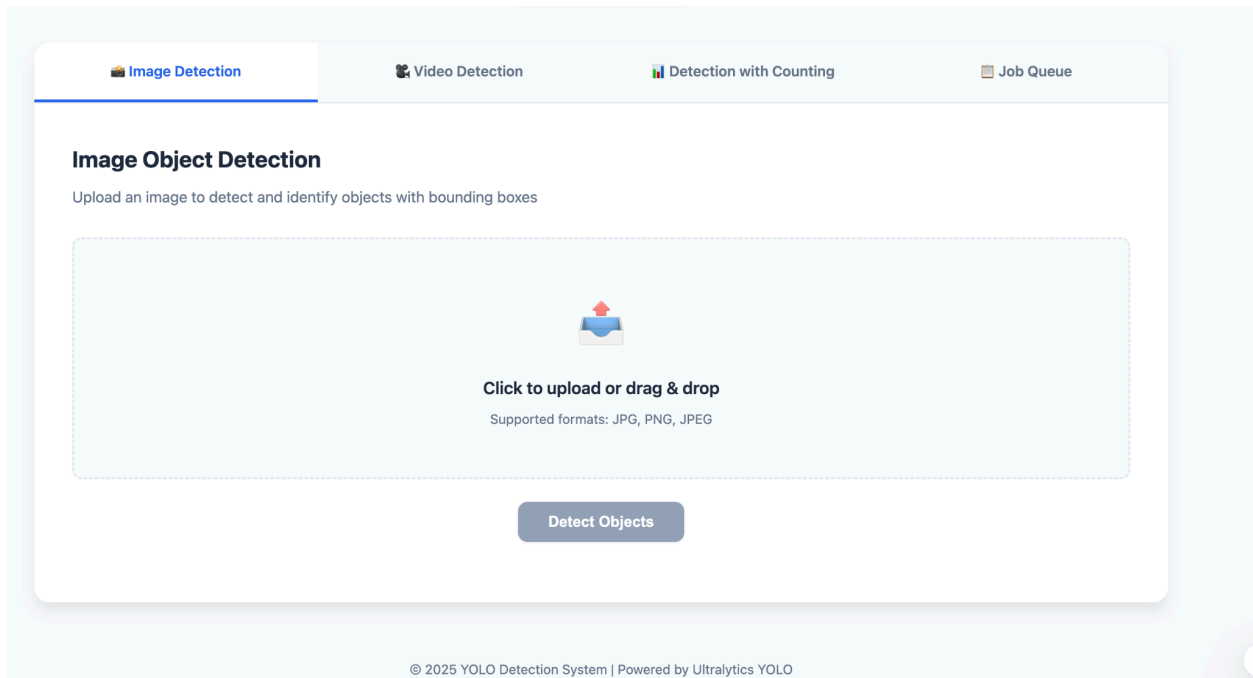


Live Camera Detection

Real-time object detection using your device camera with instant results

- ✓ Real-time camera feed processing
- ✓ Live object detection & counting
- ✓ Interactive statistics panel
- ✓ Export to PDF, DOCX, or Excel
- ✓ Mobile device support

[Start Live Detection](#)



5.2 Tracking Integration

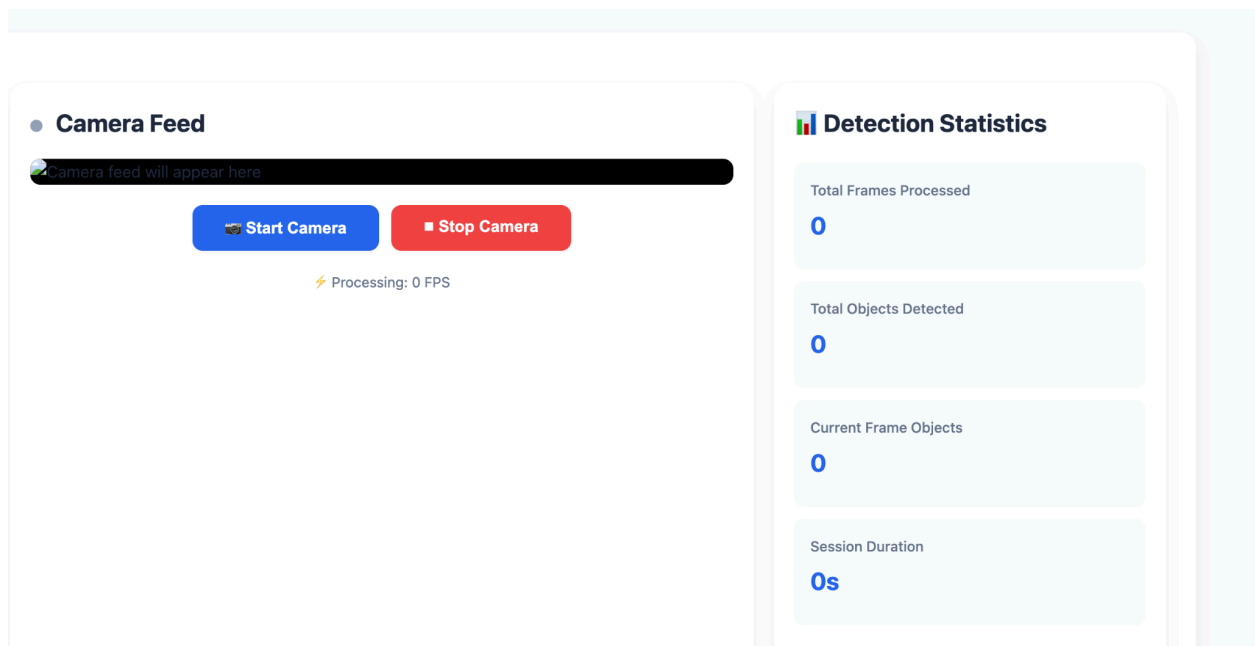
Added API support for tracking:

- `/image_inference`
- `/video_inference`
- `/tracking_inference`

📸 [Add image here — terminal output or API log screenshot]

5.3 Live Camera Streaming

Integrated **OpenCV** with Flask for live camera access.
Enabled **real-time object detection and counting**.



5.4 SSL and Security Setup

- Solved camera access issue due to HTTP restrictions.
- Installed **SSL certificates** and upgraded server to **HTTPS**.
- Result: Secure, stable live access and full API functionality.

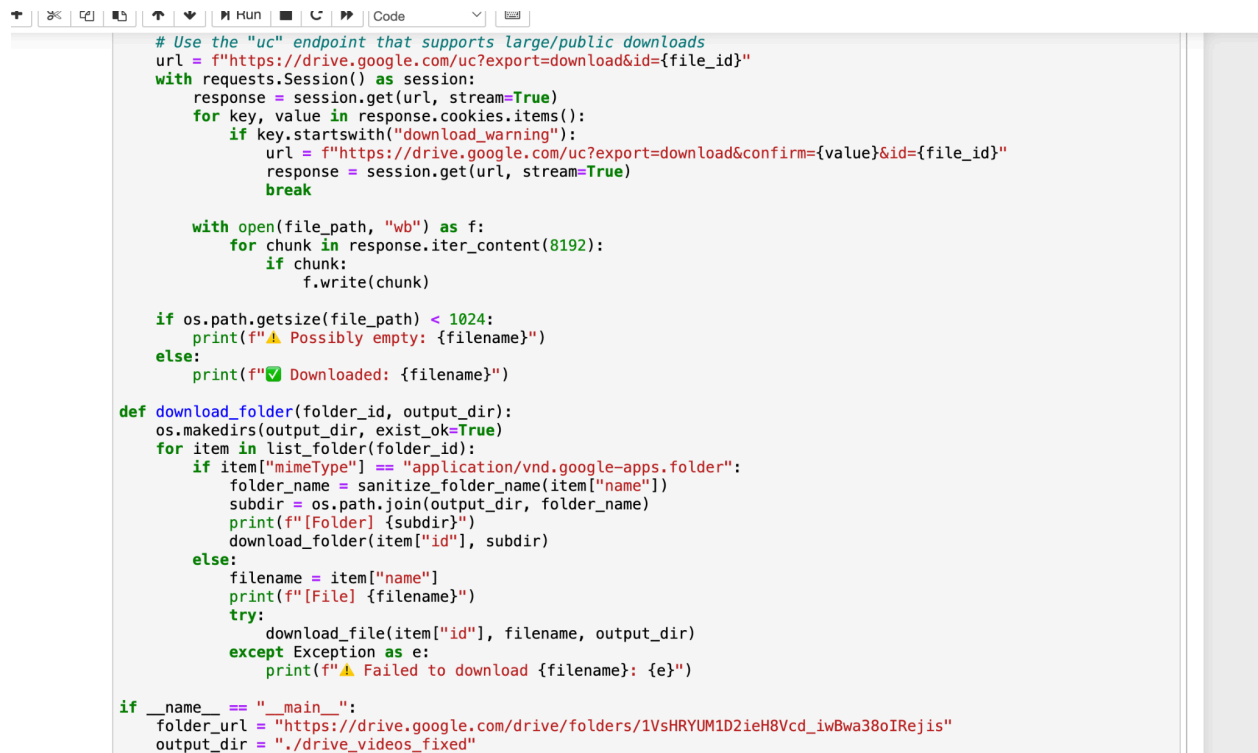
6. Workflow Automation

6.1 Objective

Automate repetitive manual work — from video downloads to frame uploads — to save time and reduce human effort.

6.2 Automated Video Download

- Integrated **Google Drive API** for automatic video downloads.
- Added retry logic for API rate limits.



```
# Use the "uc" endpoint that supports large/public downloads
url = f"https://drive.google.com/uc?export=download&id={file_id}"
with requests.Session() as session:
    response = session.get(url, stream=True)
    for key, value in response.cookies.items():
        if key.startswith("download_warning"):
            url = f"https://drive.google.com/uc?export=download&confirm={value}&id={file_id}"
            response = session.get(url, stream=True)
            break

    with open(file_path, "wb") as f:
        for chunk in response.iter_content(8192):
            if chunk:
                f.write(chunk)

if os.path.getsize(file_path) < 1024:
    print(f"⚠️ Possibly empty: {filename}")
else:
    print(f"✅ Downloaded: {filename}")

def download_folder(folder_id, output_dir):
    os.makedirs(output_dir, exist_ok=True)
    for item in list_folder(folder_id):
        if item["mimeType"] == "application/vnd.google-apps.folder":
            folder_name = sanitize_folder_name(item["name"])
            subdir = os.path.join(output_dir, folder_name)
            print(f"[Folder] {subdir}")
            download_folder(item["id"], subdir)
        else:
            filename = item["name"]
            print(f"[File] {filename}")
            try:
                download_file(item["id"], filename, output_dir)
            except Exception as e:
                print(f"⚠️ Failed to download {filename}: {e}")

if __name__ == "__main__":
    folder_url = "https://drive.google.com/drive/folders/1VsHRYUM1D2ieH8Vcd_iwBwa38oIRejis"
    output_dir = "./drive_videos_fixed"
```

```
if __name__ == "__main__":
    folder_url = "https://drive.google.com/drive/folders/1VsHRYUM1D2ieH8Vcd_iwBwa38oIRejis"
    output_dir = "./drive_videos_fixed"
    folder_id = get_folder_id_from_url(folder_url)
    download_folder(folder_id, output_dir)
    print("🟢 All done.")
```

```
[Folder] ./drive_videos_fixed/hand_warmers__24291164081__11
[File] 20251031_115333.mp4
🟢 Downloaded: 20251031_115333.mp4
[File] 20251031_115340.mp4
🟢 Downloaded: 20251031_115340.mp4
[File] 20251031_115352.mp4
🟢 Downloaded: 20251031_115352.mp4
[File] 20251031_115406.mp4
🟢 Downloaded: 20251031_115406.mp4
[File] 20251031_115314.mp4
🟢 Downloaded: 20251031_115314.mp4
[Folder] ./drive_videos_fixed/gaucha_white__80692950517__1
[File] 20251031_115212.mp4
🟢 Downloaded: 20251031_115212.mp4
[File] 20251031_115228.mp4
🟢 Downloaded: 20251031_115228.mp4
[File] 20251031_115234.mp4
🟢 Downloaded: 20251031_115234.mp4
[File] 20251031_115240.mp4
🟢 Downloaded: 20251031_115240.mp4
```

6.3 Frame Extraction Automation

- Script automatically extracts frames from each video.
- Verified frame quality and intervals automatically.

 [Add image here — extracted frames preview]

6.4 Roboflow Upload Automation

- Automated upload of frames using **Roboflow API**.
- Ensures fast and consistent dataset updates.

```
upload_data_to_roboflow_api Last Checkpoint: a day ago (autosaved)
File Edit View Insert Cell Kernel Help Not Trusted |envy O
In [10]: import os, glob, time, random, requests
         from pathlib import Path
         from tqdm import tqdm

In [11]: type_name = 'zingers_raspberry__888109010089_6'

In [12]: # ===== CONFIG =====
         API_KEY = "EG5BE30T0zEXlkeuR5hn" # your Roboflow API key
         DATASET_NAME = "products_annotations-rc8gz" # your dataset slug
         FOLDER = f"/home/ubuntu/additional_drive/shwan_data/scripts/all_frames/{type_name}" # path to image folder
         IMAGE_PATTERN = "*.jpg" # *.jpg / *.png / *.g etc.
         SEED = 42
         MAX_RETRIES = 3
         DELAY_BASE = 1.0
         TAG_WITH_FOLDER = True # adds folder name as Roboflow tag
         # =====

         upload_url = f"https://api.roboflow.com/dataset/{DATASET_NAME}/upload"
         session = requests.Session()
         random.seed(SEED)

In [13]: r = requests.get("https://api.roboflow.com/", params={"api_key": API_KEY})
         print(r.status_code, r.text)

200 {
  "welcome": "Welcome to the Roboflow API.",
  "instructions": "You are successfully authenticated.",
  "docs": "https://docs.roboflow.com",
  "workspace": "annotations-swude"
}

In [14]: IMG_EXTS = {".jpg", ".jpeg", ".png", ".bmp", ".tif", ".tiff", ".webp"}
```

```
In [16]: successes, failures = 0, 0
tags = [folder.name] if TAG_WITH_FOLDER else None

for split_name, split_files in [("train", train), ("valid", valid), ("test", test)]:
    print(f"\nUploading {split_name} ({len(split_files)} files)...")
    for p in tqdm(split_files):
        ok, resp = upload_one(p, split_name, tags=tags)
        if ok:
            successes += 1
        else:
            failures += 1
            print(f"[FAIL] {p.name} -> {resp}")

print("\n=== Summary ===")
print(f"Successes: {successes}")
print(f"Failures : {failures}")
```

Uploading train (48 files)...

100%|██████████| 48/48 [05:44<00:00, 7.17s/it]

Uploading valid (6 files)...

100%|██████████| 6/6 [00:43<00:00, 7.26s/it]

Uploading test (6 files)...

100%|██████████| 6/6 [00:46<00:00, 7.82s/it]

```
=== Summary ===
Successes: 60
Failures : 0
```

6.5 Unified Automated Pipeline

✅ Full automation achieved:

Download → Extract → Upload → Annotate → Train

7. Automatic Annotation Research

7.1 Objective

To reduce manual effort and speed up the labeling process using AI-driven automatic annotation models.

7.2 Experiments and Findings

During this month, we tested three different models for automatic annotation:

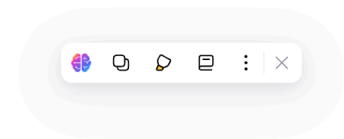
1. Ultralytics SAM (Segment Anything Model)

- Focuses on segmentation-based annotation.

- Generates object masks automatically.
- Works well for known or pre-defined classes.
- Limitation: Struggles with new or unseen product types.

Speed: 1.2ms preprocess, 14.7ms inference, 1.4ms postprocess per image at shape (1, 3, 640, 384)

Segmentation Overlay: mini_croissant



2. CLIP (Text-Prompt Based Annotation)

- **Works on text-to-image understanding.**
- Can annotate images using text prompts like “detect black-colored objects.”
- Offers flexibility but accuracy was low for our product-specific images.
- Performs better on general datasets rather than small, unique product sets.

Masks overlayed for prompts:
7_days_mini_croissants product packet





3. DINO + SAM (Combined Model)

- Combines DINO's feature extraction with SAM's segmentation ability.
- Provides better object boundaries and mask quality.
- Works effectively even on new product types without predefined labels.
- Still improving, but currently gives the best results among all tested models.

```
cv2.imwrite(f"mask_{i+1}.png", mask_uint8)
print(f"Mask {i+1} saved as 'mask_{i+1}.png'")
```

Detected 2 object(s): ['##te', '##te']
Confidence scores: [0.34128046 0.26813284]



Annotated image saved as 'annotated_output.jpg'
Number of masks generated: 2
Mask 1 saved as 'mask_1.png'
Mask 2 saved as 'mask_2.png'

In []:

Summary

After testing all three models, DINO + SAM showed the most promising results — offering accurate object detection, flexible adaptability to new products, and high-quality segmentation masks.

8. Current Focus and Next Steps

8.1 Ongoing Work

- Fine-tuning **DINO + SAM** for semi-auto annotation
 - Integrating automation with **Roboflow**
 - Improving YOLO accuracy with new datasets
-

8.2 Upcoming Goals

- Fully automate the **data-to-model workflow**
- Deploy tracking and counting system in live environments
- Optimize inference speed and lower latency